

1. 数组最大值与最小值算法分析

(a) 证明该算法的正确性

为了证明算法的正确性，使用循环不变式来进行形式上的证明。在外层 **for** 循环的每一轮迭代（循环变量为 i ）开始前，变量 x 和 y 分别是子数组 $A[1..2i-1]$ 的最大值和最小值。

初始化：在循环第一轮开始之前（即 $i = 1$ 时）， $x = A[1]$ 且 $y = A[1]$ 。此时子数组为 $A[1..1]$ ，显然 x 和 y 分别是该子数组的最大值和最小值，不变式成立。

保持：在第 i 轮循环中，算法首先比较新引入的两个元素 $A[2i]$ 和 $A[2i+1]$ 。

- 如果 $A[2i] > A[2i+1]$ ，算法会将较大的 $A[2i]$ 与当前最大值 x 比较更新，将较小的 $A[2i+1]$ 与当前最小值 y 比较更新。
- 反之亦然。这样，每次只用3次比较就能将2个新元素正确地放入排好序的数组中。因此，在第 i 轮迭代结束（即第 $i+1$ 轮迭代开始前）， x 和 y 分别更新为了 $A[1..2i+1]$ 的最大值和最小值，循环不变式保持成立。

终止：当循环结束时， $i = n+1$ 。根据循环不变式，此时的 x 和 y 分别是子数组 $A[1..2n+1]$ （即整个数组 A ）的最大值和最小值。算法输出正确。

(b) 比较运算的总执行次数

在 **for** 循环中，每次迭代首先执行一次比较 $A[2i] > A[2i+1]$ 。随后，无论进入 **then** 分支还是 **else** 分支，都会各自严格执行两次额外的比较操作（一次与 x 比较，一次与 y 比较）。

因此，每次循环固定执行 $1 + 2 = 3$ 次比较。循环总共执行 n 次，总比较次数为：

$$T(n) = 3n$$

2. 冒泡排序算法分析

(a) 还需要证明什么？

除了需要证明该算法能终止且输出序列满足 $A'[1] \leq A'[2] \leq \dots \leq A'[n]$ 外，还需要证明输出序列 A' 是输入序列 A 的一个排列（即包含了完全相同的元素）。

(b) 循环不变式

1. 内层循环不变式（变量 j）：在每次内层循环迭代开始时， $A[j]$ 是子数组 $A[j..n]$ 中的最小元素。

初始化：在循环的第一轮迭代开始之前， $j=n$ 。此时子数组 $A[j..n]$ 仅包含一个元素，即 $A[n]$ 。它自然是该单元素子数组中的最小元素，不变式成立。

保持：如果在某次迭代开始前它是正确的，即 $A[j]$ 是 $A[j..n]$ 中的最小元素。在循环体内，算法比较 $A[j]$ 和 $A[j-1]$ ，若 $A[j]<A[j-1]$ 则交换两者的值。这确保了经过比较后， $A[j-1]$ 成为了原 $A[j-1..n]$ 范围内的最小元素。随后变量 j 递减为 $j-1$ 。因此，在下一次迭代开始前，新的 $A[j]$ 依然是新的子数组 $A[j..n]$ 中的最小元素，不变式得以保持。

终止：内层循环在 $j=i$ 时结束。根据不变式，此时的 $A[i]$ 是子数组 $A[i..n]$ 中的最小元素，根据保持性 $A[1..n]$ 有序，即所需的输出。

2. 外层循环不变式（变量 i）：在每次外层循环迭代开始时，子数组 $A[1..i-1]$ 包含原数组中最小的 $i-1$ 个元素，且已经按从小到大的顺序排好序。

初始化：在循环的第一轮迭代开始之前， $i=1$ 。此时子数组 $A[1..0]$ 是一个空数组。空数组可以被认为包含了原数组中最小的 0 个元素，且是平凡有序的，不变式成立。

保持：如果在某次迭代开始前它是正确的，即 $A[1..i-1]$ 包含全局最小的 $i-1$ 个元素且有序。根据内层循环的终止性质，本次外层循环迭代结束后，内层循环会将剩余子数组 $A[i..n]$ 中的最小元素放置在位置 i 上。因为 $A[1..i-1]$ 已经是全局最小的元素，所以这个新的 $A[i]$ 必然是全局第 i 小的元素。随着外层循环准备进入下一次迭代， i 增加 1，子数组扩展为 $A[1..i]$ ，它依然包含原数组中最小的 i 个元素且有序，不变式得以保持。

终止：外层循环在 $i=n$ 时结束。根据不变式，此时子数组 $A[1..n-1]$ 包含了原数组中最小的 $n-1$ 个元素，且已经排好序。既然前 $n-1$ 个较小的元素都已经就位，那么原数组中剩下的最后一个元素 $A[n]$ 必定是全局最大的元素。因此，整个数组 $A[1..n]$ 已经完全按从小到大的顺序排好序，从而证明了算法是正确的。

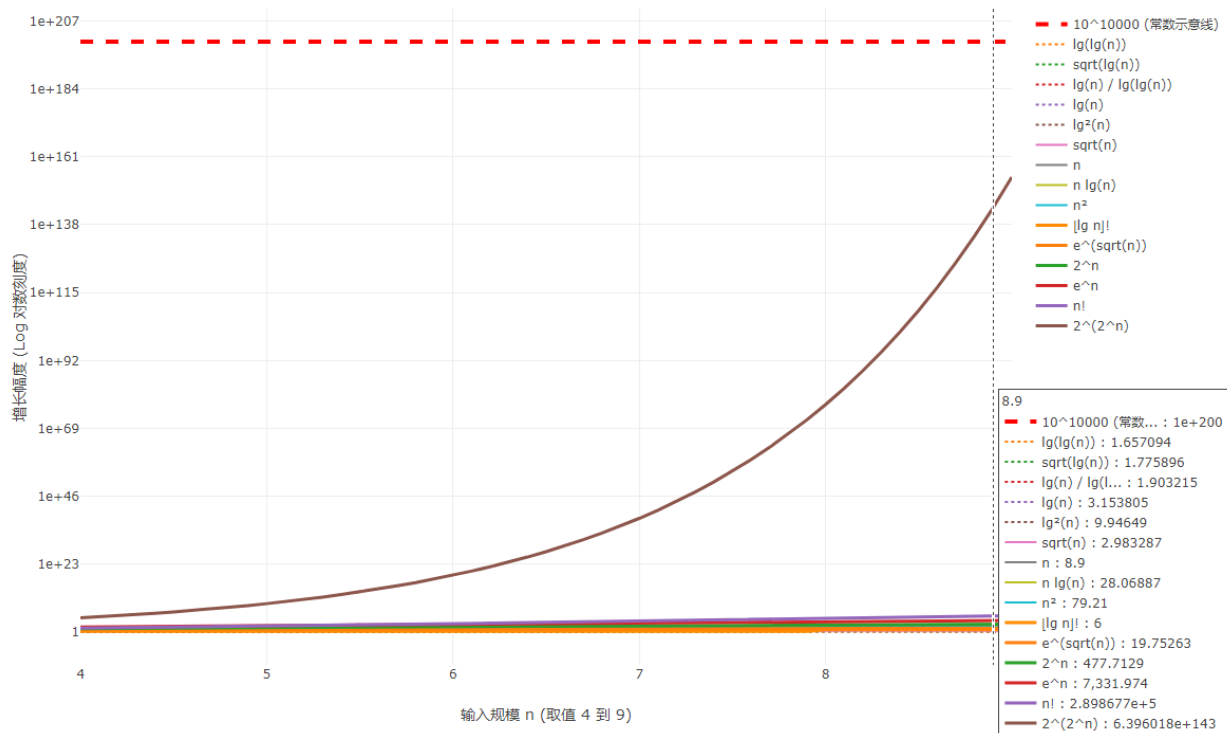
(c) 最坏情况运行时间及与插入排序的比较

最坏情况运行时间：冒泡排序的最坏情况发生在输入数组完全逆序时。此时内层循环每次都需要进行交换，总比较和交换次数为 $\sum_{i=1}^{n-1} (n-i) = \frac{n(n-1)}{2}$ 。忽略低阶项和常数系数后，其最坏情况运行时间用渐进阶表示为 $\Theta(n^2)$ 。

与插入排序的比较：插入排序的最坏情况运行时间同样是 $\Theta(n^2)$ 。但在平均情况和最佳情况下（如数组基本有序），插入排序只需进行极少的比较和移动，而冒泡排序的常数因子通常更大且交换操作更频繁，因此在实践中插入排序的效率通常优于冒泡排序。

3. 函数渐进增长速度排序

$$10^{10000} < \lg \lg n < \sqrt{\lg n} < \lg n / \lg \lg n < \lg n < \lg^2 n < \sqrt{n} < n < n \lg n < n^2 < \lfloor \lg n \rfloor! < 2^n < n^{\sqrt{n}} < e^n < n! < 2^{2^n}$$



4. 收集卡片问题的期望分析

令总卡片种类为 n 。期望收集到 $k = \lceil 0.99n \rceil$ 种卡片。

定义随机变量 X_i 为在已经收集到 $i - 1$ 种不同卡片后，直到收集到第 i 种新卡片所需要购买的干脆面数量。 X_i 服从几何分布，其每次抽到新卡片的概率为 $p_i = \frac{n-(i-1)}{n}$ 。

其期望值为 $E[X_i] = \frac{n}{n-i+1}$ 。

总期望购买包数 $E[X]$ 为各阶段期望之和：

$$E[X] = \sum_{i=1}^{\lceil 0.99n \rceil} E[X_i] = n \sum_{i=1}^{\lceil 0.99n \rceil} \frac{1}{n-i+1} = n \sum_{j=n-\lceil 0.99n \rceil+1}^n \frac{1}{j}$$

利用调和级数 $H_n \approx \ln n$ 近似：

$$\sum_{j=0.01n}^n \frac{1}{j} \approx \ln n - \ln(0.01n) = \ln \left(\frac{n}{0.01n} \right) = \ln 100$$

因此，总期望购买包数为 $n \ln 100$ 。忽略常数项后，期望需要购买的干脆面数量用渐进阶表示为 $\Theta(n)$ 。

5. 随机序列模拟投掷硬币

不能。

理由：独立投掷 n 次均匀硬币不仅要求每一次硬币出现反面（记为0）的边际概率为 0.5，还严格要求这 n 次事件是互相独立的。题干中给出的随机算法仅仅保证了每一个单独的位置 $P(A[i] = 0) = 0.5$ （边际概率），但并未说明 $A[i]$ 之间是否独立。例如，该算法可能随机生成一个 0 或 1，然后将整个数组 A 的所有元素都赋值为这个数字；这样虽然满足了题目中所有 i 都有 0.5 的概率为 0，但这 n 个结果是完全强相关的，显然无法用来模拟 n 次独立的硬币投掷结果。

6. 球与盒子问题分析

(a) 互相独立性与两两独立性

不是互相独立的随机变量。因为只有 $\lfloor \log_2 n \rfloor$ 个基础的独立随机变量 X_j 。随着 n 的增大，多个 Y_i 的值必然是由相同的一组 X_j 线性组合决定的。例如， Y_4 (由 $X_1 + X_2$ 决定) 的结果完全可以由 $Y_2 (X_1)$ 和 $Y_3 (X_2)$ 的结果推导出来，因此它们不能是互相独立的。

是两两独立的随机变量（除去固定的 Y_1 ）。对于任意两个不同的球 u 和 v ($u, v > 1$)，它们的编号 $u - 1$ 和 $v - 1$ 在二进制表示上至少有一位不同，假设第 k 位不同。那么在计算 Y_u 和 Y_v 的模 m 加和时，其中一个包含了 X_k ，另一个不包含。由于 X_k 是独立且均匀分布在 $\{1..m\}$ 的，因此在已知其中一个盒子编号的情况下，另一个盒子的编号由于有额外的未知的 X_k 参与，依然是均匀分布在 $\{1..m\}$ 中的。

(b) 计算 Z 的数学期望

定义指示器随机变量 $I_{u,v}$ ：当第 u 个球和第 v 个球落入同一个盒子时 $I_{u,v} = 1$ ，否则为 0。因为 Y_u 和 Y_v 是两两独立的且均匀分布在 m 个盒子中，它们落入同一个盒子的概率为：

$$P(Y_u = Y_v) = \frac{1}{m}$$

因此 $E[I_{u,v}] = \frac{1}{m}$ 。总对数 Z 可以表示为所有组合的指示器变量之和：

$Z = \sum_{1 \leq u < v \leq n} I_{u,v}$ 。根据期望的线性性质：

$$E[Z] = \sum_{1 \leq u < v \leq n} E[I_{u,v}] = \binom{n}{2} \cdot \frac{1}{m} = \frac{n(n-1)}{2m}$$

(c) 证明 $m = n^2$ 时，最多只包含一个球的概率大于 $1/2$

当 $m = n^2$ 时，代入 (b) 中求得的期望：

$$E[Z] = \frac{n(n-1)}{2n^2} = \frac{n-1}{2n}$$

因为 $n \geq 1$ ，显然有 $\frac{n-1}{2n} < \frac{n}{2n} = \frac{1}{2}$ 。即 $E[Z] < 1/2$ 。

“每个盒子中最多只包含一个球”这一事件等价于“投入同一个盒子的球的对数 $Z = 0$ ”。

因为 Z 是一个非负整数随机变量，根据马尔可夫不等式，有：

$$P(Z \geq 1) \leq E[Z] < \frac{1}{2}$$

因此， $Z = 0$ 的概率为：

$$P(Z = 0) = 1 - P(Z \geq 1) > 1 - \frac{1}{2} = \frac{1}{2}$$

得证。